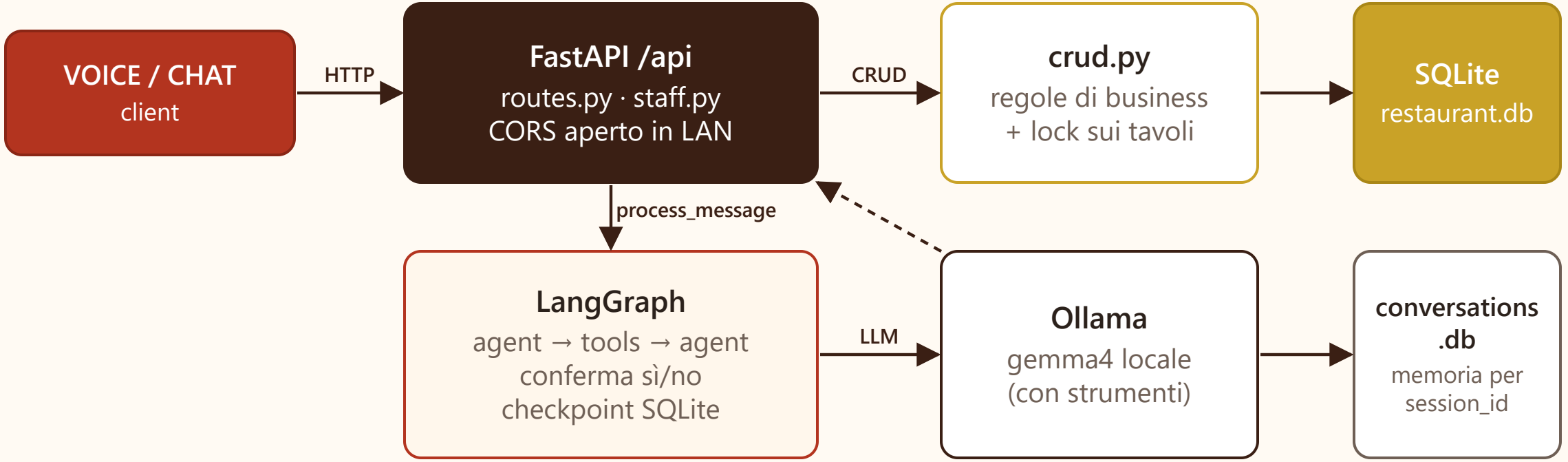
PRESENTAZIONE TECNICA

Il BE e dove il Voice lo incontra

Cosa fa il backend, quali endpoint usiamo, cosa succede dentro a ogni richiesta e cosa chiedere al team BE

Il BE in una pagina



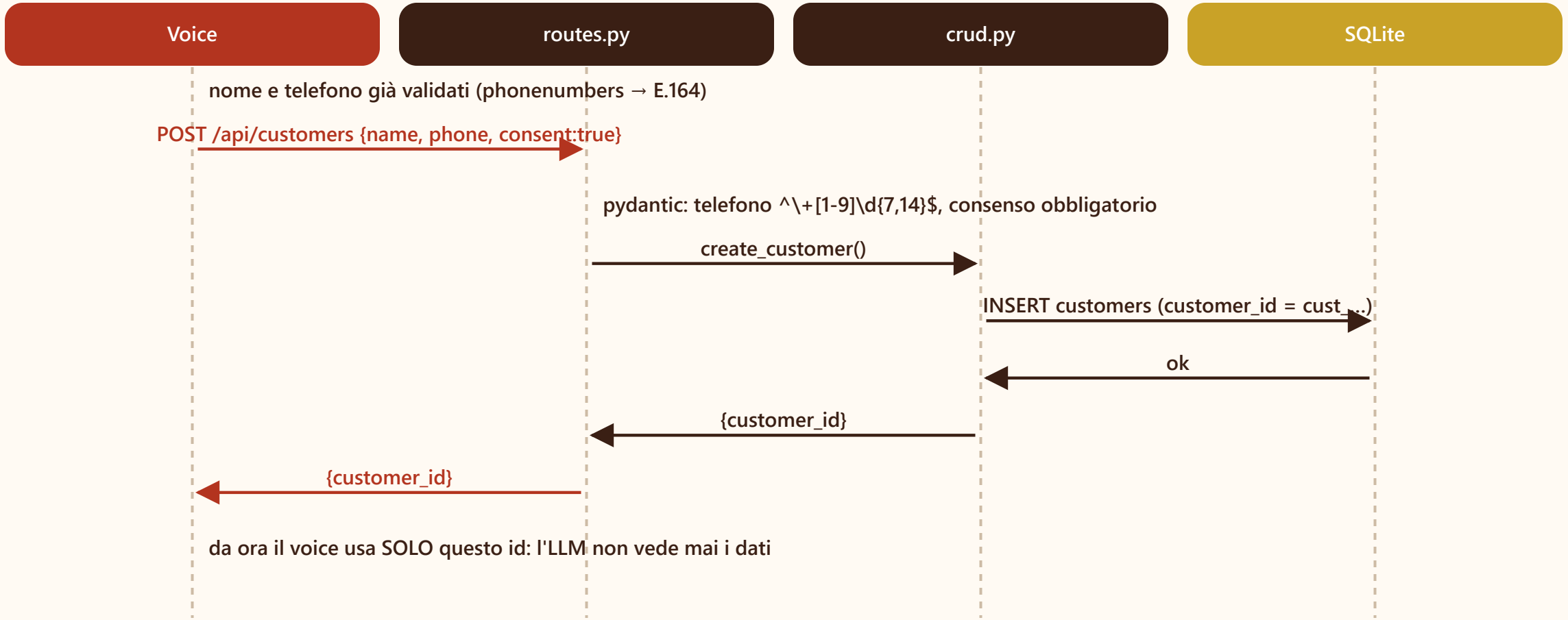
Il BE è l'unico che parla con l'LLM e con il database: i client sono «sottili».

Dove il Voice tocca il BE

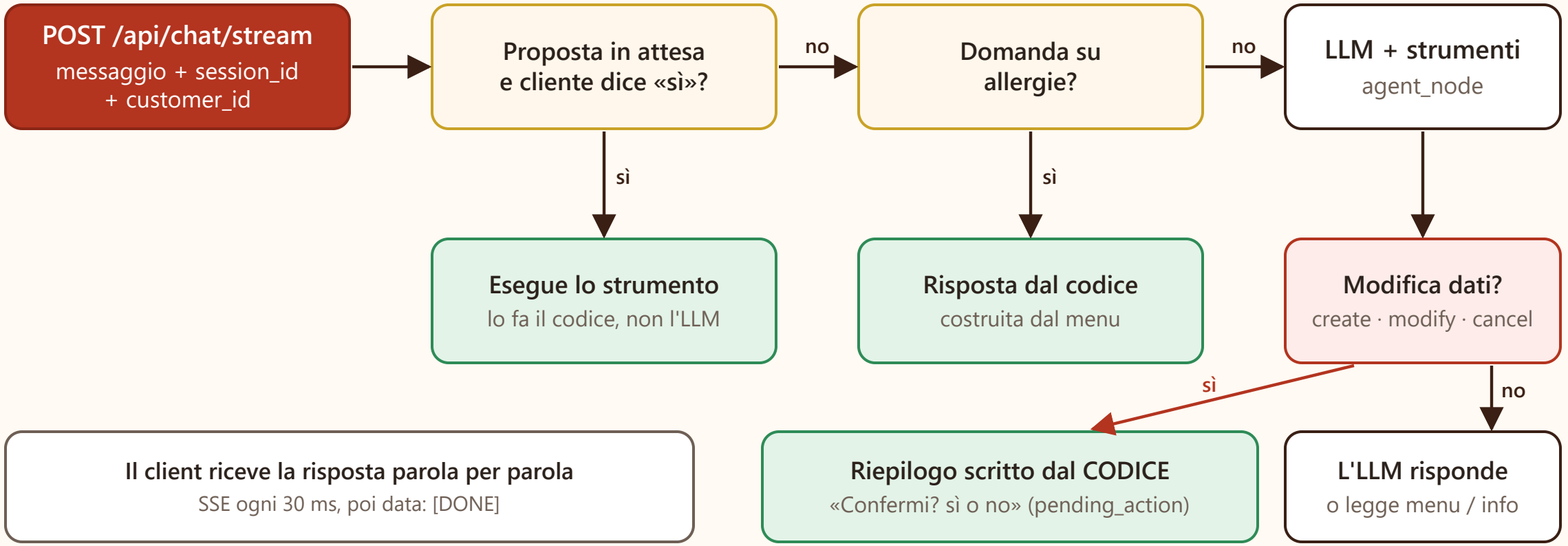
Nel codice voice	Endpoint del BE	Cosa succede
`BackendClient.health_info()`	GET `/api/health` (poi `/health`)	database e modello rispondono davvero: online / degraded
`register_customer()`	POST `/api/customers`	salva nome+telefono, restituisce `cust_…` (unico punto con dati personali)
`chat()`	POST `/api/chat`	un turno intero (usato da v2 e dai referti)
`stream_chat()`	POST `/api/chat/stream`	stessa cosa ma a token SSE; il BE **prima** esegue tutto il turno, poi manda le parole
`get_reservation()`	GET `/api/reservations/{codice}?customer_id=`	solo il proprietario: gli altri ricevono 404
server voice `/site/info`	GET `/api/info`	indirizzo, telefono, orari per il sito
server voice `/site/menu`	GET `/api/menu`	menu con filtri (vegano, senza glutine…)

Tutto passa da `voice/src/voicebot/backend.py` (httpx, retry, timeout) · il browser non parla mai direttamente col BE.

Registrazione del cliente: l'unica volta con dati personali



Dentro un turno di chat (backend/graph/workflow.py)



Il protocollo di conferma: l'LLM propone, il codice esegue

Esempio

```
Cliente  vorrei un tavolo per due domani alle 20
LLM      → propone create_reservation(...)
CODICE   «Riepilogo: tavolo per 2 persone sabato 17
          ottobre alle 20. Confermi? sì o no»
Cliente  sì
CODICE   esegue lo strumento → codice RES-9A8B2C

Cliente  sì, ma alle 21     ← NON è una conferma:
         il BE torna all'LLM come modifica
```

Perché è fatto così

- Un modello debole creava prenotazioni **fantasma** su saluti e frasi senza senso
- Una richiesta impossibile (giorno di chiusura, orario fuori turno, gruppo troppo grande) **non viene nemmeno proposta**
- `is_confirmation()`: vale solo se il messaggio è fatto **solo** di parole di conferma (≤ 4)
- Il voice fa lo stesso a valle: il suo referto finale e il suo «sì» sono **un secondo livello** di conferma, a voce

Dati e regole di business

tables	reservations	customers	menu_items	closures · info
table_number	booking_code (RES-…)	customer_id (cust_…)	category · name · price	closures: data + motivo
capacity (posti)	customer_id	name	allergens	restaurant_info:
location: 3 ambienti	table_id	phone (E.164)	flag: veg · vegan · GF	chiave → valore
is_active	date · time · shift	consent_at	available (esaurito)	
	guests_count			
	status prenotazione			

Regole in crud.py

turno di 90 minuti: un tavolo è occupato se le finestre [ora, ora+90] si sovrappongono · tavolo più piccolo che basta (best fit)
pranzo 12:00–15:00 (ultima 14:00), cena 19:00–23:30 (ultima 22:00) · chiuso il lunedì · lock sull'allocazione dei tavoli

L'LLM nel BE: scelta e velocità

Scelta del modello (llm_provider.py)

- `OLLAMA_MODEL` può essere un elenco in ordine di preferenza + `OLLAMA_FALLBACK_MODELS`
- Si usa il **primo installato che sa usare gli strumenti** (`/api/show` → capabilities): senza strumenti non si prenota
- Nessun modello adatto → 503 con un messaggio chiaro, non un errore tecnico
- La scelta è nel log e in `/api/health`

Velocità e robustezza

- **Thinking spento** di default (3–6 volte più veloce): `OLLAMA_REASONING`
- **Warm-up** all'avvio in un thread: il modello è già in memoria al primo cliente
- `OLLAMA_KEEP_ALIVE=30m`, timeout 120 s, temperatura 0,1
- `python -m backend.check_model`: 8 prove con OK/KO e tempi
- `health` = «online» solo se database E modello rispondono

Il prompt del BE e la cache dell'LLM

Regole fisse e lunghe
ruolo, stile, menu solo da strumento, divieti

«Cliente NON registrato»
blocco che cambia con la registrazione (righe 20–22)

Data odierna
cambia ogni giorno (riga 43)

customer_id
cambia per cliente (righe 48–50, 72)

Istruzioni degli strumenti
usano il customer_id

Misurato su un portatile
primo turno di un cliente: 41–75 s · turni successivi: 1–5 s
il modello legge ~36 token/s, scrive ~7 token/s

Proposta per il BE
mettere data, stato di registrazione e customer_id
in FONDO al prompt: la parte lunga resta in cache

Cosa fa il Voice nel frattempo
id anonimo condiviso anon_voice · warm-up all'avvio ·
contesto limitato · nuova sessione ogni 20 scambi

Oggi: appena cambia una riga rossa, l'LLM rilegge tutto il resto (cache persa).

Cosa chiedere al team BE

Priorità	Richiesta	Perché
Alta	L'endpoint `/api/chat/stream` è `async` ma chiama `process_message` in modo bloccante	mentre l'LLM lavora il BE non risponde a nulla, nemmeno a `/api/health` (osservato: timeout di 10–20 s)
Alta	Spostare data / registrazione / customer_id in fondo al prompt	cache dell'LLM intatta, primo turno per cliente da ~60 s a pochi secondi
Alta	Il modello a volte scrive «Chiamo get_menu…» invece di chiamare lo strumento	il voice lo filtra, ma la causa è nel prompt/modello del BE
Media	Allineare il prompt vocale a `voice/src/voicebot/prompts.py`	oggi il BE legge solo l'ultimo messaggio e usa il suo prompt
Media	Impostare `num_ctx` esplicito per il modello	evita contesti da 64k su macchine con poca RAM
Media	Cognome solo se c'è già una prenotazione simile	serve vedere le prenotazioni di altri: solo il BE può farlo, nel rispetto della privacy
Bassa	Endpoint «prenotazioni del cliente» (`/api/customers/{id}/reservations`)	la pagina potrebbe elencarle tutte, non solo quelle di questo browser